



DevVault

AI-Powered Developer Knowledge Vault

DEVELOPMENT TEAM:

Devansh Singh & Project AI Assistant

DOCUMENT SCOPE:

System Architecture, Google Gemini AI Orchestration, API Endpoints, and Database Layout

RELEASE DATE:

July 2026

Abstract: DevVault is a full-stack, single-page application (SPA) serving as a premium developer knowledge base and note-taking vault. Utilizing custom vanilla CSS styled around a modern slate design system, the application embeds Google Gemini AI to offer code-snippet explanation modules, auto-categorizations, and interactive conversational developer mentorship.

1. System Architecture Overview

DevVault leverages a decoupled Single-Page Application (SPA) architecture, matching a modular client-server model communicating via a RESTful API pipeline. All network payloads are encrypted and formatted in JSON.

Frontend Client Layer

The user interface is powered by React (Vite environment) and structured using custom vanilla CSS rules, establishing a consistent dark-theme/light-theme tokenization without third-party styling frameworks. Framer Motion is integrated to manage physics-based interface animations, page transitions, and drawer panels. Axios serves as the HTTP client proxy, featuring a request interceptor to append JWT bearer tokens to outbound authorization headers.

Backend Service Layer

The application server is an Express API application built on Node.js. It manages token authorization, request payload validations, custom category registration states, and proxies requests to the Google Gemini AI software development kit (SDK).

Database Persistence Layer

Data persistence is handled by MongoDB, a document-based database. Relationships are modeled using mongoose schemas. A startup script drops legacy database text indexes (like duplicate username keys) and compiles query-optimized indexes targeting card search parameters (Title, Categories, Tags, and Languages).

2. Google Gemini AI Orchestration

DevVault utilizes the Google Gemini API to embed advanced large language model capabilities. To protect developer keys, all requests are proxied through server-side controller middleware rather than calling the API directly from the browser bundle.

AI Feature set:

Suggest appropriate: categories and tags dynamically while creating or editing notes. It reads title/description inputs and aligns recommendations with the user's custom categories.

Analyze code snippets: and descriptions, generating clear markdown breakdowns that identify time/space complexity ($O(n)$) and engineering practices.

A sliding chat drawer: loaded with note contexts. Users can chat with the AI about the note's implementation. Gemini acts as an expert software mentor and rejects off-topic queries.

Example Server-Side AI System Prompt Configuration:

```
const prompt = `You are an expert software engineer helping another developer understand a technical note.
Context Note:
Title: ${card.title}
Description: ${card.description}
Category: ${card.category}
Code Snippet (${card.language}): ${card.codeSnippet}

User Question: ${question}

Instructions:
1. Answer the user's question concisely in the context of this card.
2. If the user's question is completely unrelated to the card, politely reject it.
3. Answer in clear, concise markdown. Avoid conversational fluff.`;
```

3. Database Layout & Schema Models

DevVault maintains three core Mongoose collections: Users, Notes, and Categories. The relations are modeled with document references.

User Schema Model (models/User.js)

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 6 },
  title: { type: String, default: "" },
  description: { type: String, default: "" },
  profilePhoto: { type: String, default: "" }
}, { timestamps: true });
```

Note Schema Model (models/Card.js)

```
const noteSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true, trim: true },
  description: { type: String, required: true },
  codeSnippet: { type: String, default: "" },
  language: { type: String, default: "" },
  category: { type: String, required: true, trim: true },
  tags: [{ type: String }],
  isFavourite: { type: Boolean, default: false },
  isPinned: { type: Boolean, default: false },
  visibility: { type: String, enum: ['local', 'global'], default: 'local' },
  createdBy: { type: String, default: "Anonymous" },
  favouritedBy: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  favouritedCount: { type: Number, default: 0 }
}, { timestamps: true });
```

4. API Endpoints Reference

The backend exposes endpoints grouped by feature area, secured using JWT auth headers.

Authentication Routes (/api/auth)

POST /register Registers a user account. Hash password and returns user details + JWT token.

POST /login Validates credentials and returns user details + JWT token.

GET /me Fetches authenticated user's profile information.

PUT /profile Updates profile name, job title, and bio description.

POST /upload-photo Uploads user avatar image via multer multipart/form-data, deleting old file.

Notes Routes (/api/cards)

GET / Gets user's notes (includes owned notes and favorited community notes).

POST / Creates a new local or community note.

GET /global Queries public community notes shared by all developers (supports sorting/search).

GET /id Retrieves details of a single note by ID.

PUT /id Updates details of a note. Restricts updates to original creator.

DELETE /id Deletes a note. Restricts deletions to original creator.

AI Services Routes (/api/ai)

POST /category-and-tag Generates category and tag suggestions for a draft note.

POST /deepig Retrieves deepIG Gemini AI code explanation breakdowns.

POST /mentor Submits a query to the AI Mentor chatbot context drawer.

